Training–Rollout Mismatch in Low-Precision LLM Reinforcement Learning: A Survey

Zunhai Su
The University of Hong Kong

Abstract: Low-precision rollout generation can reduce the cost of language-model reinforcement learning while changing the policy that supplies its training data. Numerical approximation then propagates through gradient estimation, policy updates, and subsequent generation. We survey twenty focal studies through this feedback loop. A common formulation separates execution mismatch from policy staleness and decoding transformations, and distinguishes conditional correction from response-level change of measure. A component taxonomy connects numerical alignment and fidelity control with estimator correction, update control, and schedule adaptation. Three relationships emerge: the effectiveness of correction evolves with training; alignment depends on the computation shared across a specific interface; and signed update contributions can expose effects hidden by aggregate discrepancy. These relationships explain how numerical and learning interventions complement one another, and why closer agreement alone does not establish better learning. We compare their assumptions, information requirements, and system costs, and identify open problems in predictive diagnosis, composition, and stateful collection.

Keywords: quantized rollout; low-precision reinforcement learning; training–inference mismatch; importance sampling; quantization-aware training.

Literature snapshot: October 19, 2026.

Survey resources: https://github.com/ZunhaiSu/awesome-low-precision-rl

Living survey, under active development. Suggestions, corrections, and research collaborations are welcome via GitHub Issues.

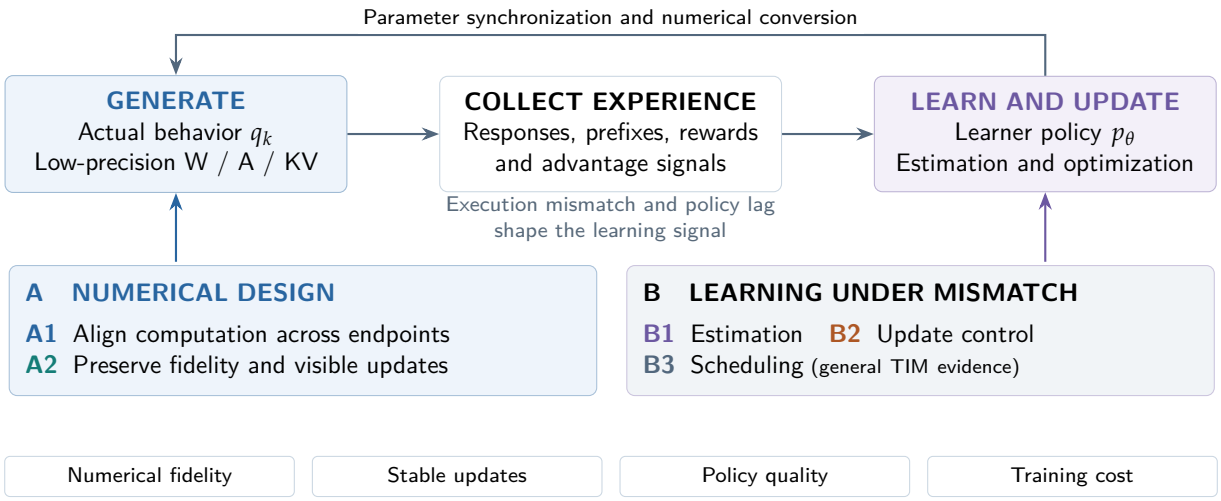


Figure 1: Survey at a glance. Numerical methods act on learner or sampler execution; learning methods change how samples drive updates. Their effects return through the rollout–learning loop.

Contents

1 Introduction and Scope **4**

1.1 Why Low-Precision Rollouts Complicate Policy Optimization 4

1.2 Scope and Related Work 4

1.3 Perspective and Contributions 5

2 Problem Formulation and Taxonomy **6**

2.1 Learner, Behavior, Proximal, and Reference Policies 6

2.2 Execution Mismatch versus Policy Staleness 7

2.3 A Component-Level Taxonomy of Interventions 8

2.4 Numerical Regimes and Contribution Roles 9

3 Mechanisms: From Numerical Discrepancy to Learning Dynamics **9**

3.1 Sources and Propagation of Execution Discrepancy 9

3.2 Distorted Estimation and Update Decisions 10

3.3 Feedback across Optimization Steps 11

3.4 Diagnosing the Effects of Mismatch 12

4 Numerical Design and Execution Alignment **12**

4.1 Aligning Learner and Rollout Computation 12

4.2 Controlling Quantization Error and Preserving Parameter Updates 13

4.3 Residual Mismatch and Limits of Alignment 14

5 Learning under Residual Mismatch **15**

5.1 Correcting Gradient Estimation 15

5.2 Controlling the Influence of Policy Updates 16

5.3 Adapting the Optimization Schedule 17

5.4 Compatibility, Bias, and Failure Modes 17

6 Synthesis of Findings and Practical Trade-offs **18**

6.1 Experimental Regimes 18

6.2 Findings on Stability and Learning Quality 19

6.3 Efficiency Gains and Implementation Costs 20

6.4 Limitations of Current Evidence 22

7 Open Problems **22**

7.1 Predicting Harmful Mismatch 22

7.2 Composing Numerical and Learning Interventions 22

7.3 Precision, Staleness, and Long-Horizon Interaction 23

7.4 Generalization and Comparable Evidence 23

8 Conclusion **23**

A Surveyed Papers and Section Mapping **24**

B Source Selection and Version Handling **25**

C Evidence, Assumptions, and Transfer **26**

1. Introduction and Scope

1.1. Why Low-Precision Rollouts Complicate Policy Optimization

In language-model reinforcement learning, generation determines the responses, rewards, and histories from which the next policy is learned. Quantizing the rollout engine changes this source of training data as well as its computational cost. A local numerical difference can alter a sampled token, the learning signal assigned to a response, and the parameter update that shapes later generation. The resulting training–rollout mismatch is therefore part of a feedback process whose consequences extend beyond the accuracy of a single forward pass.

The practical objective is to learn a capable policy within a finite computational budget. Faster generation contributes to that objective through the quality of the data it supplies and the updates those data support. Numerical agreement, optimization stability, task performance, and runtime capture different parts of this relationship. Aggressive clipping, for example, can stabilize training while discarding useful signal; a more faithful sampler can preserve that signal while reducing the throughput gained from quantization. Their value depends on how efficiently the complete learning process improves the policy.

Existing methods intervene at different points in this process. FlashRL uses actual rollout probabilities to correct learning from a different sampling policy; Jet-RL coordinates learner and sampler precision flow; subsequent studies act on quantization, clipping, gradient estimation, and local update dynamics [1, 3]. These interventions often coexist within one system. Classifying their components makes the relationship visible: numerical design determines the data-generating computation, while learning-side rules determine how the resulting samples influence the next policy.

Figure 2 organizes these components by the computation or update they change. Cross-category assignments preserve the distinct contributions of integrated methods; Appendix A summarizes the corresponding papers.

1.2. Scope and Related Work

This survey focuses on numerical and execution mismatch in LLM reinforcement learning, especially quantized rollout and related low-precision training configurations. It covers a higher-precision learner paired with a lower-precision sampler, as well as low precision on both sides. Weight, activation, and key–value (KV) cache quantization affect different computational interfaces. Policy staleness enters where parameter changes interact with those interfaces.

The twenty focal contributions comprise thirteen method or system studies, six formulation or general mismatch studies, and one exploration boundary case. These roles describe their use in the synthesis: MiniRL, for example, develops the formulation and also evaluates FP8 rollout [14]. The literature snapshot is October 19, 2026. Appendix B describes source selection and version handling.

Broader surveys address low-precision LLM training, rollout strategies, and reinforcement learning for reasoning models [24–26]. The distinctive concern here is the connection between actual sampling probabilities, numerical execution, and the update computed from sampled data. General post-training quantization, pretraining-only methods, and speculative-draft acceleration enter only where they clarify that connection.

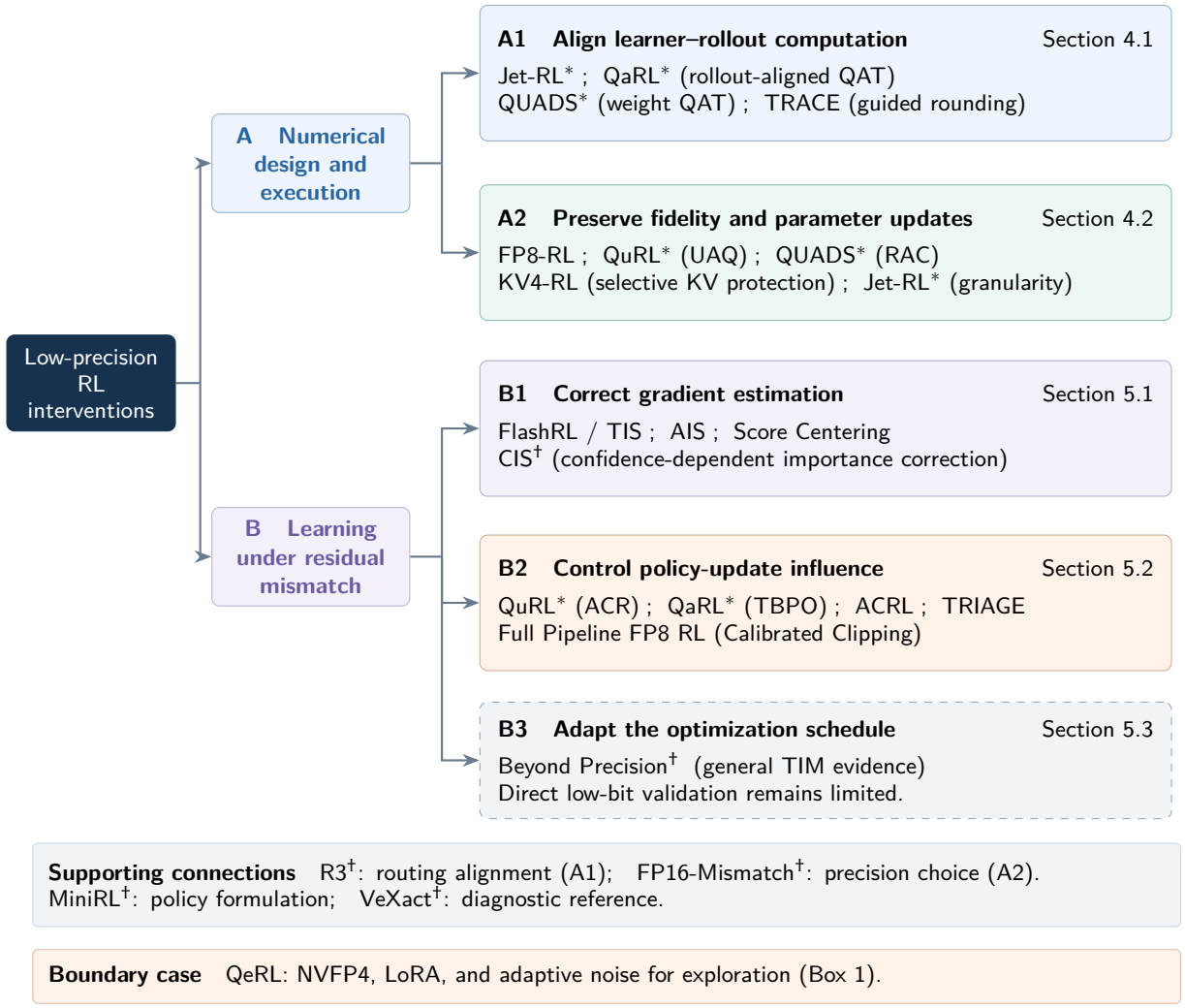


Figure 2: Taxonomy of the surveyed work. Branches identify what each component directly changes; repeated names reveal cross-category contributions. All thirteen core studies, six supporting studies, and the exploration boundary case are shown. Numerical configurations remain separate comparison axes. Appendix A gives the paper-by-paper summary and discussion locations.

The collection supports a focused narrative synthesis of primary studies. Numerical formats, architectures, adaptation regimes, and collection schedules remain separate from the method taxonomy, so that similar mechanisms can be compared without treating unlike experimental settings as equivalent.

1.3. Perspective and Contributions

The central perspective is that numerical fidelity and learning correction are coupled resources. A more faithful sampler changes the errors that correction must handle; a different update rule changes which numerical errors are consequential. The quantity of interest is the learning signal preserved by their joint design, together with the computation needed to preserve it.

We develop this perspective by connecting policy definitions, failure mechanisms, and method components. The formulation in Section 2 identifies which distribution each probability ratio refers to; Section 3 traces discrepancies through estimation and optimization. This foundation supports the comparison of numerical and learning interventions in Sections 4–5. Their shared dependencies explain the published findings and practical trade-offs synthesized in Section 6, from which the open problems in Section 7 follow.

2. Problem Formulation and Taxonomy

2.1. Learner, Behavior, Proximal, and Reference Policies

Let x be a prompt, $y_{1:T}$ a response, and $h_t = (x, y_{<t})$ its prefix. A policy is specified by both a parameter state and the computation used to evaluate it. Write $p_\theta(a \mid h)$ for the current learner policy and $p_k(a \mid h)$ for the same learner-side path at sampling snapshot θ_k.

The behavior policy $q_k(a \mid h)$ is the distribution that actually sampled the token. It includes the rollout engine, numerical representation, and sampling transformation. Temperature and vocabulary truncation therefore belong to its definition: a raw-model log-probability returned by an engine need not equal the probability after decoding transformations.

The collection protocol is part of the behavior law. Under deterministic execution and version schedules, the generated history determines the sampler state, including cache entries computed under earlier parameter versions. The subscript k abbreviates that rollout path; a checkpoint identifier alone cannot specify stateful execution.

A proximal policy p_{prox} supplies the denominator of a policy-update constraint. A reference policy p_{ref} supplies a regularization target. These are different roles: a frozen pre-update learner may be proximal, while a much earlier policy is the KL reference. Neither is automatically the policy that generated every token. PPO and GRPO provide familiar contexts for these distinctions [21, 22]; mismatch-aware formulations require them to remain explicit [1, 14].

Table 1: Policies and quantities used throughout the survey. Execution paths are part of policy identity.

Symbol	Meaning
p_θ	Current learner policy under its specified execution path.
p_k	Learner-side policy evaluated at sampling snapshot θ_k.
q_k	Actual behavior policy that sampled the token.
p_{prox}	Policy used to define the proximal update ratio.
p_{ref}	Policy used for KL regularization; not the behavior denominator.
Δ_k	Same-snapshot log-probability difference, $\log p_k - \log q_k$.
d_θ	Current learner–behavior gap, $\log p_\theta - \log q_k$.
A_t	Advantage or other policy-gradient learning signal.
$s_\theta(a, h)$	Score vector, $\nabla_\theta \log p_\theta(a \mid h)$.

2.2. Execution Mismatch versus Policy Staleness

At a fixed prefix and token with positive relevant probabilities,

$$\frac{p_\theta(a \mid h)}{q_k(a \mid h)} = \underbrace{\frac{p_\theta(a \mid h)}{p_k(a \mid h)}}_{\text{parameter change}} \underbrace{\frac{p_k(a \mid h)}{q_k(a \mid h)}}_{\text{same-snapshot execution/decoding difference}} . \tag{1}$$

The first factor captures parameter change since collection, including asynchronous version lag; the factorization requires no independence assumption. The second compares execution at the sampling snapshot. It isolates numerical differences when decoding is matched and also includes decoding differences when temperature or top-p filtering changes. The intermediate policy p_k is a conceptual reference and need not be retained by the implementation. In partial rollouts, $k = k(t)$ may vary across tokens. If caches survive synchronization, their earlier computations remain part of the behavior path, so reconstructing q_k requires cache provenance or the realized sampling probabilities as well as the current checkpoint.

Figure 3 separates these snapshot comparisons from the two policy roles used by the loss. A proximal anchor and a KL reference can be chosen independently of the actual sampler.

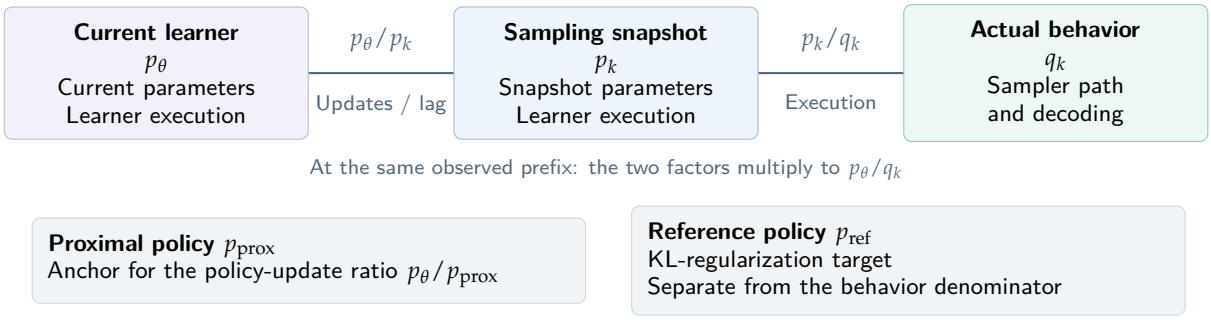


Figure 3: Policy identities and loss roles. Horizontal links denote the ratio comparisons in Equation (1), rather than data flow. Execution differences isolate numerical mismatch when decoding is matched. The lower cards identify separate roles in optimization.

Two limiting cases make the distinction concrete. At the sampling snapshot, the learner-side parameter-change factor is one while execution mismatch can remain. With matched execution and decoding, the second factor can instead be one while policy lag remains. A numerical intervention and a staleness correction therefore solve different parts of the same ratio.

Probability ratios also require support conditions. A token observed from q_k has a defined denominator, but that alone does not establish that q_k covers every action with positive target probability. Truncation sampling can violate this condition. Similarly, a token-level change of measure is not automatically a correction for the prefix distribution. At fixed h,

$$\mathbb{E}_{a \sim q_k(\cdot \mid h)}\left[\frac{p_\theta(a \mid h)}{q_k(a \mid h)} f(a,h)\right] = \mathbb{E}_{a \sim p_\theta(\cdot \mid h)}[f(a,h)] \tag{2}$$

holds under support and integrability assumptions. It says nothing by itself about whether the outer distribution over h is the desired one. It also does not undo changes to group-relative advantages induced by collecting different responses.

For comparison, the full-response likelihood ratio is

$$W_\theta(y \mid x) = \frac{P_\theta(y \mid x)}{Q(y \mid x)} = \prod_{t=1}^{T} \frac{p_\theta(y_t \mid h_t)}{q_{k(t)}(y_t \mid h_t)}. \tag{3}$$

Here P_θ and Q are the induced response laws, and the denominator factors must be their actual conditional probabilities under the documented protocol. Unobserved execution randomness requires marginalizing that state or extending the likelihood to the corresponding execution events. The two response laws use a common stopping convention; an end-of-sequence decision is included when applicable. For a fixed behavior law and a parameter-independent reward R, the usual likelihood-ratio identity gives

$$\nabla_\theta \mathbb{E}_{P_\theta}[R] = \mathbb{E}_Q\left[W_\theta R \sum_{t=1}^{T} s_\theta(y_t, h_t)\right] \tag{4}$$

under support, integrability, and differentiation assumptions. The product illustrates why exact response-level correction can have high variance; multiplying only the current token ratio into an arbitrary sampled advantage is a different estimator. Nor does Equation (4) directly justify reusing group-normalized advantages computed under Q. Score Centering explicitly distinguishes sequence-level importance sampling from practical token-level corrections [10].

A two-step example in Figure 4 makes the distinction tangible. The second-token conditionals agree exactly, yet the first-token histories have different probabilities. A correction using only the second-token ratio therefore leaves that prefix imbalance, whereas the full-response ratio includes the earlier decision.

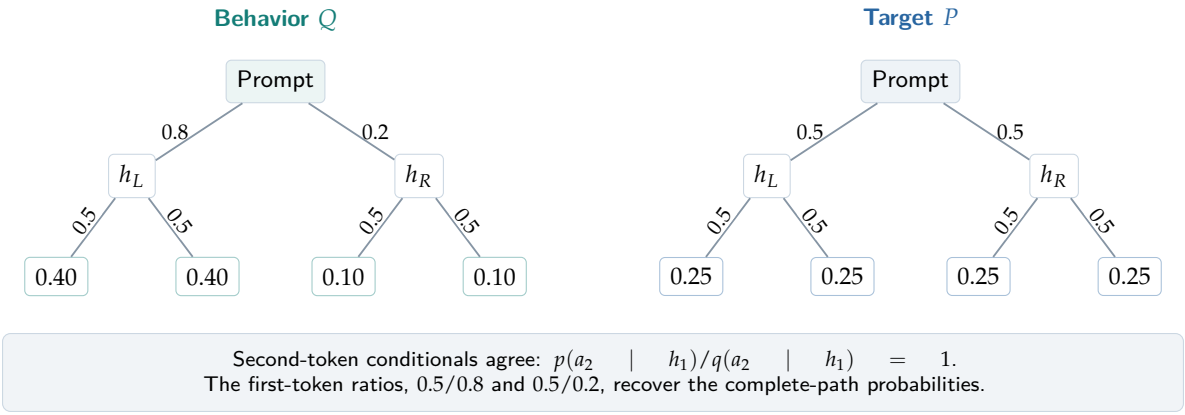


Figure 4: Exact two-step illustration of conditional and response-level correction. Leaf values are complete-response probabilities. Equal second-token conditionals coexist with different prefix masses; the product ratio restores the four target probabilities of 0.25. These are constructed probabilities, not experimental results.

2.3. A Component-Level Taxonomy of Interventions

We group components by the object they directly modify. **A: numerical interventions** alter representation or execution. A1 matches computation between learner and sampler; A2 controls quantization error or preserves parameter updates. **B: learning interventions** alter how sampled data produce updates. B1 corrects gradient estimation; B2 controls the acceptance, magnitude, or direction of policy updates; B3 adapts the optimization schedule.

The classes follow each component's operative rule and design objective. An alignment method may also improve fidelity, and every estimator correction ultimately changes an update. The useful distinction is what the component is constructed to do. A multiplicative weight, for example, can correct a distributional expectation or regulate update influence; its algebraic form alone does not determine its class. B3 extends the taxonomy to schedule adaptation, for which direct low-bit evidence remains limited.

Table 2: Five intervention classes. These categories describe operative components, not exclusive paper types.

Class	Direct object	Representative components	Residual limitation
A1	Shared computation	Precision flow; aligned QAT; guided rounding	Unmatched execution interfaces.
A2	Quantization fidelity	UAQ; residual compensation; calibration; selective precision	Resolution of useful parameter changes.
B1	Gradient estimator	TIS; AIS; CIS; Score Centering	Target expectation and approximation bias.
B2	Update influence	ACR; TBPO; calibrated clipping; direction-aware controls	Retained signed learning signal.
B3	Optimization schedule	Mismatch-informed LR adaptation	Stability relative to learning progress.

2.4. Numerical Regimes and Contribution Roles

The same intervention can operate under different W/A/KV precisions, dense or mixture-of-experts (MoE) architectures, low-rank or full-parameter updates, and synchronous or partial collection. Native kernels and quantize–dequantize simulation also implement different numerical paths. In particular, "FP4 training" leaves open the precision of master weights, backward computation, and optimizer states.

A study's role in the taxonomy depends on its contribution, while its experiments determine where that contribution has been tested. A formulation study can contain quantized-rollout evidence, and a quantized system can integrate an existing correction. This distinction preserves the experimental scope and attribution of both.

3. Mechanisms: From Numerical Discrepancy to Learning Dynamics

3.1. Sources and Propagation of Execution Discrepancy

Scale selection, rounding, accumulation, kernel choice, batch shape, and cached state can all alter a numerical policy's output distribution. Two paths using the same nominal format can consequently disagree. FP16-based analysis illustrates the role of precision selection, while R3 exposes the effect of discrete MoE routing decisions [15, 16]. Routing disagreement can also arise independently of quantization.

Shared-prefix comparison holds the token history fixed and isolates differences between conditional distributions. Free-running comparison allows different samples to change the histories visited later. These settings expose different stages of propagation: a small conditional error can redirect generation, while a favorable shared-prefix average can conceal disagreement on histories reached by only one policy.

Discretization introduces a further asymmetry. An update can be visible in the learner's parameter space but fail to change the sampler's quantized representation. Conversely, a small change near a rounding boundary can abruptly change a codeword. QuRL studies the former through update-aware quantization; TRACE targets inconsistency in rounding outcomes [5, 13]. The relevant object is therefore not only error magnitude, but its relationship to the evolving parameters and the discretization boundaries.

Figure 5 organizes the resulting learning effects into relative influence, expected direction, and update selection. These routes can coexist, which explains why one aggregate disagreement statistic can miss the active mechanism.

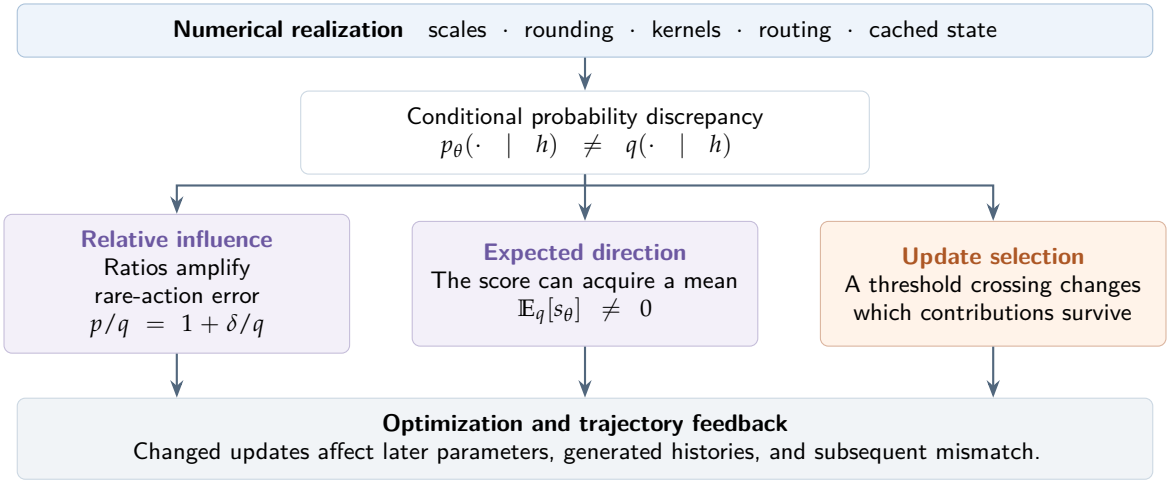


Figure 5: Three routes from execution discrepancy to learning dynamics: relative influence, expected score direction, and update selection. These effects can coexist and feed back through later updates and generated histories. Arrows indicate possible propagation under the policy and estimator assumptions in the text.

3.2. Distorted Estimation and Update Decisions

At an observed prefix, numerical disagreement can change an update through its weight, its expected direction, or the clipping branch it activates. Absolute probability error captures only part of this effect. Writing $p = q + \delta$ gives $p/q = 1 + \delta/q$: the same absolute perturbation has greater relative influence when q is small. Tails and confidence-conditioned statistics therefore reveal structure that a mean absolute error can hide. CIS uses this confidence dependence in its correction design [20].

Estimation can also acquire a systematic drift. For a normalized differentiable learner distribution, $\mathbb{E}_{a\sim p_\theta}[s_\theta(a,h)] = 0$. Under behavior sampling, the corresponding expectation need not vanish. At fixed h,

$$\mathbb{E}_q[A s_\theta] = \mathrm{Cov}_q(A, s_\theta) + \mathbb{E}_q[A]\mathbb{E}_q[s_\theta]. \tag{5}$$

The expectations are conditional on the prefix and include the sampled action and any random continuation used to form A. Centering advantages across a prompt group need not center them at each prefix. Score Centering uses this distinction to motivate an additive correction [10]: it removes a first-moment drift term while leaving the behavior distribution itself unchanged.

Mismatch additionally affects decisions made by a surrogate objective. In a clipped policy-gradient objective, changing a ratio can change which branch is active and therefore whether a token contributes gradient. Thus, a perturbation can matter through a threshold crossing even when its

absolute magnitude appears modest. The full-pipeline FP8 study isolates a failure involving excessive clipping of negative-advantage tokens [11]. This is a different intervention target from merely decreasing the variance of a sampled importance weight.

The sign dependence can be seen without assuming a particular quantizer. Let $\ell(r, A) = \min\{rA, \mathrm{clip}(r, 1-\epsilon, 1+\epsilon)A\}$, with $0 < \epsilon < 1$ and fixed A. Away from the two boundaries,

$$\frac{\partial \ell}{\partial r} = \begin{cases} A\,\mathbf{1}\{r < 1+\epsilon\}, & A > 0, \\ A\,\mathbf{1}\{r > 1-\epsilon\}, & A < 0, \\ 0, & A = 0. \end{cases} \tag{6}$$

Thus the active clipping side depends on the learning signal. For an illustrative objective that clips the combined learner–behavior ratio, a same-snapshot ratio of 0.7 with $\epsilon = 0.2$ already suppresses a negative-advantage token, even before a learner update. This is not the decoupled surrogate in Section 5.1; the example explains why naming the clipped ratio is essential to interpreting a failure or a repair.

Figure 6 makes this sign dependence explicit for the standard surrogate. It also shows why clipping the combined learner–behavior ratio can suppress an update that a proximal ratio would still permit.

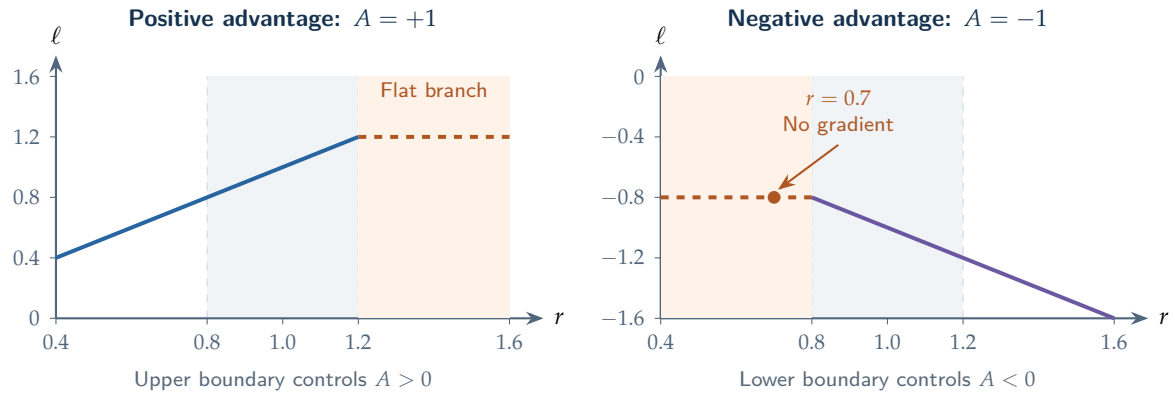


Figure 6: Signed clipping in Equation (6), with $\epsilon = 0.2$ and $A = \pm 1$. The gray interval is $[0.8, 1.2]$; amber segments are flat. At $r = 0.7$, a negative-advantage token contributes no gradient through this objective. The clipped policy ratio determines the interpretation.

3.3. Feedback across Optimization Steps

The learning loop turns a local discrepancy into a dynamic process. A sampled response changes the learner; the learner is synchronized back to the sampler; the next rollout is then generated from an updated numerical realization. Stability depends on this feedback, not solely on whether one frozen batch has a small error.

Direction matters because two updates with the same gap magnitude can move learner and behavior probabilities differently. In a simplified local picture, reducing a learner probability that is already below the behavior probability can enlarge their log gap. ACRL and TRIAGE use update-direction information in their controls [9, 12]. For a shared-parameter network, however, one token's update changes other probabilities too. The sign argument supplies local intuition; interactions through shared parameters determine the scope of any stability result.

Optimization scale adds another feedback channel. Beyond Precision studies the relationship between mismatch and optimization dynamics and proposes response-length-triggered learning-rate decay [18]. This connects numerical observations to the scale of optimization, while leaving transfer of the schedule to aggressive FP4 execution open.

3.4. Diagnosing the Effects of Mismatch

Diagnosis begins with the comparison used to isolate an effect. Replacing an inference engine can simultaneously change arithmetic, batching, memory use, and generated trajectories. A resulting quality difference cannot automatically be assigned to the precision format. VeXact constructs a zero-mismatch reference to help separate execution mismatch from other training effects [19]. It supplies a diagnostic counterfactual against which the effects of accelerated execution can be interpreted.

Execution, update behavior, and policy learning form a causal chain. Shared-prefix replay isolates changes in conditional probabilities; update diagnostics expose their effect on the estimator or constraint; training trajectories reveal the accumulated consequence. VeXact's reference is especially useful at the first interface. Connecting all three links explains why the same numerical discrepancy can be harmless under one update rule and damaging under another.

Box 1. Quantization and exploration

QeRL combines low-precision execution, low-rank adaptation, and adaptive quantization noise, making exploration an explicit part of the training design [2]. It occupies a boundary of this survey: the numerical perturbation changes the search process as well as agreement between computations. Its results establish a role for quantization noise in that setting, with task utility depending on the induced sampling distribution and the learner's adaptation capacity. They do not establish a benefit from arbitrary cross-engine error.

These mechanisms connect numerical design to the learning signal it preserves. Relative weighting, conditional drift, threshold decisions, and trajectory feedback expose different consequences of the same execution change. The numerical and learning interventions that follow address different parts of that chain.

4. Numerical Design and Execution Alignment

Numerical design offers two complementary ways to change the policy that generates training data. Alignment brings learner and sampler computations closer at a shared interface; fidelity control preserves the information that a low-precision representation would otherwise lose. The distinction matters because two approximate paths can agree without either becoming more accurate relative to a higher-precision model.

4.1. Aligning Learner and Rollout Computation

Jet-RL coordinates the precision flow used by learner and sampler [3]. Its unified FP8 design matches corresponding operators and quantization granularity across the endpoints. This makes the computation, rather than the format label, the interface of alignment.

QaRL implements such coupling through rollout-aligned quantization-aware training (QAT): the learner's forward pass reflects the representation used during generation [6]. QUADS takes a deliberately asymmetric approach, fake-quantizing learner weights while retaining higher-precision learner activations [8]. Both designs bring part of the learner computation closer to the sampler, but QUADS leaves activation error to a separate rollout-side component. Their comparison reveals that alignment can be selective; uniform precision across the whole model is neither its definition nor a prerequisite. QAT emulation and native low-precision kernels nevertheless remain distinct numerical paths.

TRACE shares realized quantization information [13]. Compact rollout-side guidance informs learner-side FP4 rounding when a common quantizer specification still yields different decisions near a boundary. Its local rounding argument explains the benefit at that interface; full-network divergence also depends on downstream computation.

The shared information distinguishes these approaches: Jet-RL coordinates precision flow, aligned QAT emulates a representation, and TRACE communicates realized numerical decisions. Routing replay couples discrete MoE choices through a related principle [15]. More specific coupling introduces dependencies on token positions, modules, and parameter versions. Agreement of the complete policies still depends on the computation surrounding the shared interface.

4.2. Controlling Quantization Error and Preserving Parameter Updates

Alignment alone does not ensure that the sampler can express the changes made by training. For an illustrative deterministic quantizer Q, an update can satisfy

$$Q(w + \delta w) = Q(w), \tag{7}$$

even when $\delta w \neq 0$. Visibility depends on the distance to a quantization boundary and on scale behavior, as well as update magnitude. QuRL's Update-Aware Quantization uses invariant scaling to alter the relationship between quantization resolution and parameter updates [5]. Its numerical contribution is therefore distinct from ACR, which modifies the clipping rule in Section 5.2.

Figure 7 illustrates why update magnitude alone does not determine visibility. The distance to the next quantization boundary can make a smaller change visible while a larger one remains within a single cell. The example holds the scale fixed; learned or recalibrated scales change the cell boundaries themselves.

QUADS addresses another residual: activation error in the rollout path. Its Residual Activation Compensation complements learner-side weight QAT, allowing alignment and error reduction to operate on different parts of the system [8]. The component ablations are particularly useful here because they distinguish what weight alignment recovers from what activation compensation adds.

FP8-RL maintains numerical fidelity as the policy changes through blockwise W8A8 rollout, scale recalibration for FP8 attention and KV execution, and dynamic weight synchronization [4]. It integrates these components with adopted TIS/MIS correction. Its v2 experiments also explore end-to-end FP8, so learner precision varies across the paper's experimental settings.

KV4-RL allocates fidelity across positions instead of raising precision uniformly [17]. INT4 KV storage is supplemented by bounded INT8 protection for attention-sink and recent positions. Its retained TIS recipe makes the distinction between sampler fidelity and conditional correction especially clear: protecting the cache changes the data-generating computation even when the correction

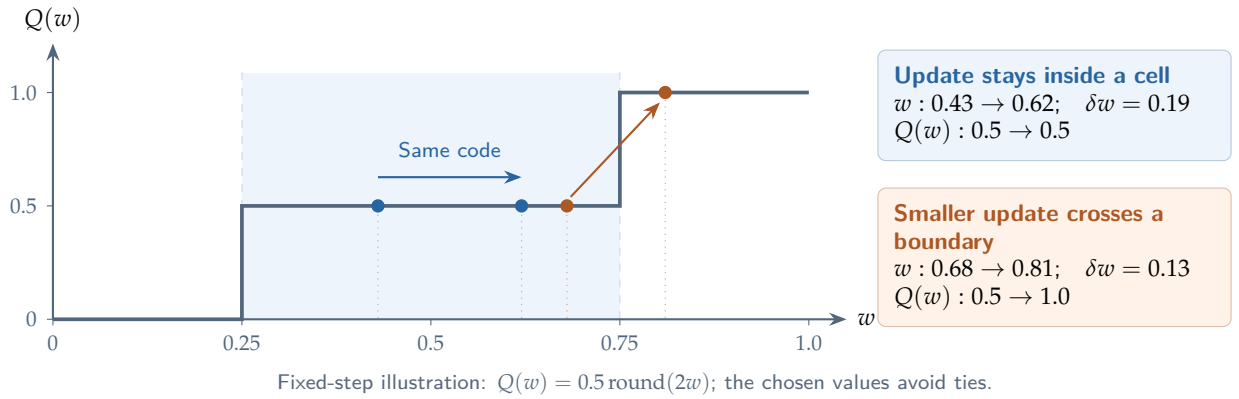


Figure 7: Constructed illustration of quantized-update visibility in Equation (7). Under the fixed-step scalar quantizer, the blue update stays inside a cell while the smaller amber update crosses a boundary. Variable scales change these boundaries.

is held fixed. The experiments use colocated, quota-based partial collection, with generation and optimization alternating; responses may span collection boundaries.

These methods preserve different information needed by learning. Parameter rescaling protects update visibility, residual compensation reduces activation error, recalibration follows distributional change, and positional protection targets sensitive cache states. Their W/A/KV targets locate the computation on which each mechanism acts.

4.3. Residual Mismatch and Limits of Alignment

Every alignment strategy leaves a boundary. Shared weights can coexist with different activations, a common quantizer with different surrounding kernels, and protected states with an otherwise unchanged network. Agreement between endpoints and fidelity to a higher-precision computation are separate properties, each conditional on the parameter snapshot and input history.

This partial coverage explains why numerical methods often coexist with learning correction. A more faithful sampler can still lag behind the learner, while a closely matched conditional distribution cannot undo the histories excluded by earlier sampling decisions. The residual determines what a learning-side intervention has left to address.

Maintaining a shared numerical interface requires information as well as arithmetic. Operator choices must stay coordinated, QAT must reproduce quantizer semantics, and rounding guidance must be stored and matched to the learner [3, 6, 8, 13]. Fidelity control instead uses calibration, compensation, or selective execution to preserve sensitive information [4, 5, 17]. Table 3 compares these dependencies and the discrepancies each leaves unresolved.

The residual mismatch determines the role of learning-side correction. Its treatment depends on whether the remaining error changes the target expectation, the influence of an update, or the evolution of optimization.

Table 3: Numerical strategies, the information they share or preserve, and their remaining execution differences.

Strategy	Shared or protected object	Information dependency	Residual discrepancy
Precision flow	Corresponding operator precision and granularity	Coordinated numerical configuration	Engine and surrounding-kernel differences.
Aligned QAT	Learner representation corresponding to rollout	Matching quantizer semantics	Native versus simulated arithmetic; unmatched activations.
Guided rounding	Realized rollout quantization decisions	Stored and matched rollout guidance	Unshared modules and downstream propagation.
Fidelity control	Scales, update resolution, residuals, or protected states	Calibration, compensation, or selective execution	Other execution paths and policy staleness.

5. Learning under Residual Mismatch

Residual mismatch enters learning through both estimation and optimization. Estimator correction changes how behavior samples represent a target expectation; update control regulates their influence; scheduling changes the scale of adaptation over time. These objectives lead to different trade-offs among bias, variance, progress, and stability.

5.1. Correcting Gradient Estimation

Multiplicative correction and truncation. Importance sampling supplies a conditional change of measure under the assumptions of Equation (2). Practical surrogate objectives usually distinguish the mismatch correction from the proximal update ratio. With frozen p_{prox} and recorded behavior q, define

$$c_t = \frac{p_{\text{prox}}(a_t \mid h_t)}{q(a_t \mid h_t)}, \qquad r_t(\theta) = \frac{p_\theta(a_t \mid h_t)}{p_{\text{prox}}(a_t \mid h_t)}. \tag{8}$$

A representative decoupled surrogate is

$$\widehat{L}(\theta) = \frac{1}{N} \sum_t \text{sg}(\bar{c}_t) \min[r_t(\theta) A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t], \tag{9}$$

where $\bar{c}_t$ is a possibly truncated correction, the sampled advantages are held fixed, and sg denotes no differentiation through the correction weight. The separation exposes two different operations: bounding the mismatch correction c_t and clipping the proximal ratio r_t. Individual objectives differ in how they perform and combine these operations. An upper-truncated correction, for example, takes $\bar{c}_t = \min(c_t, C)$, whereas masking can discard selected ratios. Here N counts valid sampled tokens. Response-level averaging and length normalization assign different relative weights, so the reduction rule is part of the estimator being compared.

FlashRL's TIS provides a basic bounded correction using actual rollout probabilities [1]. MiniRL exemplifies the mismatch-aware distinction between behavior and proximal policies underlying this notation [14]. FP8-RL adopts token-level TIS/MIS variants, while KV4-RL retains TIS alongside

cache protection [4, 17]. The system studies show how the correction operates alongside other numerical components. Truncation bounds extreme influence while changing the expectation; recording an actual behavior denominator supplies the required probability semantics, but does not remove that truncation bias.

Adapting the amount and location of correction. AIS uses observable mismatch diagnostics to interpolate between uncorrected and fully importance-weighted gradients [7]. Its adaptive quantity is the estimator mixture, not a precision schedule. CIS instead derives a confidence-dependent importance-weight cap from a discrepancy representation [20]. These approaches differ in what they adapt: overall correction strength versus the location of truncation across token confidence. CIS is used here as general TIM evidence; its BF16 cross-engine experiments are not direct FP4 validation.

Additive score correction. Score Centering replaces the score at a prefix with

$$\widetilde{s}_\theta(a,h) = s_\theta(a,h) - \mathbb{E}_{b\sim q(\cdot|h)}[s_\theta(b,h)]. \tag{10}$$

With the exact expectation, its mean under q vanishes and the drift term in Equation (5) is removed [10]. The remaining covariance is still measured under q, so this is not full recovery of an on-policy gradient. The reported implementation uses sampler top-k log-probabilities and models the omitted tail using the learner distribution; its default is $k = 128$ [10]. The omitted mass therefore enters through a modeled tail rather than being discarded. The coefficients used to form the correction are detached, and its fidelity depends on that behavior reconstruction.

The distinction between multiplicative and additive correction also determines the information returned by the sampler. Ratio-based methods need probabilities of sampled actions; exact score centering needs a behavior-weighted score expectation. Truncation and reconstruction make different approximations to these quantities. Their derivative interpretation additionally depends on the learner computation, including any surrogate gradient through a quantizer. These methods consequently offer different kinds of correction: changing relative action weights, removing a conditional drift term, or adapting how much correction is applied.

5.2. Controlling the Influence of Policy Updates

Restoring or reshaping update constraints. QuRL's Adaptive Clipping Range adjusts the update boundary in response to the relationship between proximal and behavior probabilities [5]. QaRL's TBPO combines negative-sample treatment with sequence-level trust-band control [6]. The full-pipeline FP8 method first matches the lower clipping quantile on negative-advantage tokens to a BF16 reference, then adjusts the upper boundary to rebalance positive and negative contributions [11]. Periodic reference passes amortize the calibration cost. All three act on the update constraint, but use different signals and aggregation levels. "Calibration" in the last method refers to objective boundaries, not quantizer scales.

The shared issue is which learning signal reaches the parameter update. A token-level boundary can restore a contribution lost to clipping, whereas a sequence-level trust band controls a whole response. Widening either region admits more updates, but their value depends on advantage sign and influence. The clipping count becomes informative when connected to those retained contributions.

Controlling disagreement through update direction. ACRL uses advantage-guided probability adjustment, with response-level discrepancy relative to a reference controlling token reweighting [9]. It regulates how updates change disagreement. TRIAGE combines segment-local risk information, direction-selective gating, rebalancing, and bounded repair in native NVFP4 execution [12]. Both belong to B2 because their operative rule controls update influence. Their different risk signals and aggregation levels determine which contributions are admitted, suppressed, or repaired.

The granularity of control matters. A response-level statistic aggregates evidence across many tokens; a segment statistic can expose local concentration; a token rule is directly selective but may be sensitive to noise. Aggregation trades localization against statistical reliability. Finer granularity improves localization only when the resulting diagnostic remains reliable enough to guide the update.

5.3. Adapting the Optimization Schedule

A third intervention changes the scale of learning over time. Beyond Precision motivates learning-rate scheduling through the evolution of optimization and mismatch [18]. This is complementary to changing a numerical path or a token weight: it modifies the trajectory of parameter updates themselves.

Schedule adaptation remains less developed for direct low-bit settings than the estimator and update controls above. Its promise depends on preserving progress as well as stability: smaller steps can avoid divergence while delaying useful learning. Comparable quality within a comparable budget is therefore the relevant test of transfer to aggressive quantization.

5.4. Compatibility, Bias, and Failure Modes

The operators in Figure 8 act on different objects even when they share a reweighting implementation. Importance correction targets an expectation; a gate or trust band regulates the update it permits. Their combination depends on a common behavior law and on how each operation changes the estimator received by the next.

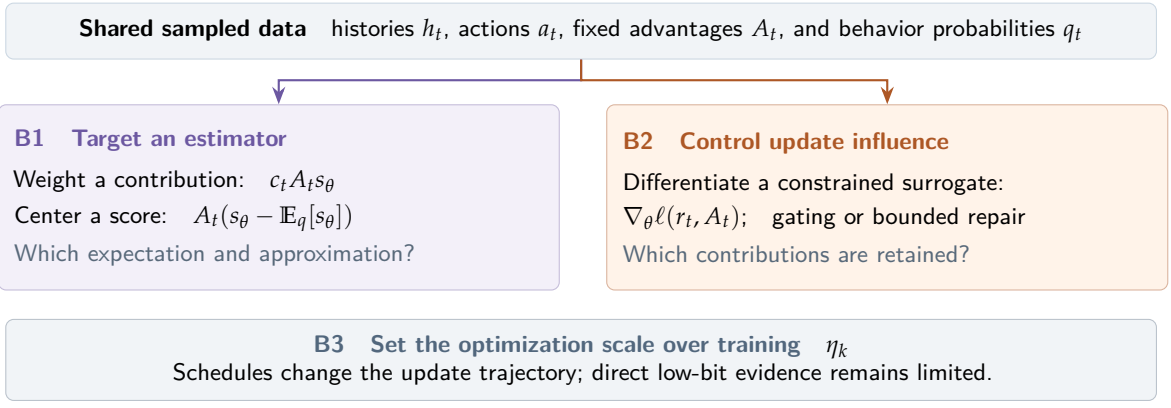


Figure 8: Learning interventions distinguished by their target object. The displayed expressions are explanatory examples with fixed sampled advantages; the B1 forms are not assumed to be interchangeable, and the B2 panel omits method-specific correction factors. A combined objective needs its own estimator definition.

Several designs already combine numerical and learning components. QaRL, QuRL, and QUADS provide evidence for the combinations studied within each system; Score Centering describes composition with importance weighting by centering the weighted score [5, 6, 8, 10]. These results motivate composition while leaving its behavior under other combinations open.

A joint rule depends on consistent behavior probabilities, parameter snapshots, and estimator definitions. Centering an unweighted score need not center a weighted one; successive masks can remove the same examples; a reference pass adds computation to an otherwise inexpensive rule. These interactions connect the statistical compatibility of components to the system costs examined next.

Composition is useful when one component addresses an error left by another. Conditional correction preserves the collected histories, numerical alignment can leave policy lag, and clipping controls influence without reproducing sampler arithmetic. Their joint benefit depends on preserving useful signal after all corrections and constraints have acted.

6. Synthesis of Findings and Practical Trade-offs

Controlled changes within individual systems reveal how numerical fidelity and learning correction interact. The resulting evidence connects three design choices: how closely the endpoints share computation, how sampled data enter the update, and which contributions survive repeated optimization. Their practical value depends on the quality gained and the system cost incurred.

6.1. Experimental Regimes

Asymmetric learner–sampler precision exposes differences between numerical representations. Low precision on both sides makes the consistency of their execution paths and the resulting update especially salient. KV quantization acts on retained state, whereas weight and activation quantization alter other parts of computation. Low-rank adaptation and full-parameter RL also provide different degrees of freedom for responding to these changes.

AIS evaluates diffusion-based LLaDA alongside autoregressive Qwen models [7]. Its autoregressive experiments connect directly to the token-level formulation used here. Interpreting the denoising results requires likelihoods defined for that generation process, so those experiments broaden the empirical scope without validating the autoregressive factorization for diffusion.

Architecture and collection protocol further condition conclusions. MoE execution introduces discrete route choices; cache-heavy long responses expose different runtime bottlenecks from shorter generations; partial collection permits a response to contain tokens sampled under different parameter versions. A method evaluated in one of these regimes may motivate a hypothesis in another, but that transfer is not itself an experimental result.

Table 4 brings together six within-study comparisons that illuminate these interactions. Appendix A covers all twenty focal works, and the companion evidence collection provides source locations for observations from all thirteen core studies.

6.2. Findings on Stability and Learning Quality

Across these regimes, the evidence links the evolution of mismatch, the interface being aligned, and the signed updates retained by the objective. Figure 9 locates these relationships across numerical measurements, update behavior, learning trajectories, and quality and cost.

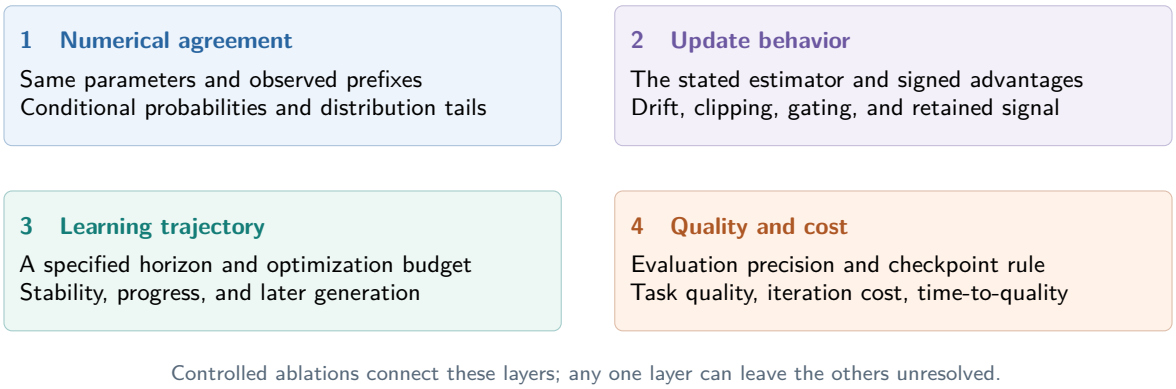


Figure 9: Four layers of evidence connecting numerical execution to effective training. Controlled ablations can link an intervention across these layers; improvement at one layer alone leaves the others unresolved.

Correction and mismatch evolve together. QuRL observes disagreement that grows beyond an initially stable period, with ACR and UAQ addressing different changes in clipping and update visibility [5]. AIS adapts the correction mixture, ACRL regulates discrepancy through update influence, and Score Centering removes a conditional estimator drift term [7, 9, 10]. These responses act on an evolving process: a correction changes the update, which changes later behavior and the discrepancy encountered by that correction. Early agreement and bounded weights therefore characterize only part of the learning trajectory.

Alignment gains depend on the shared interface. Jet-RL couples precision flow, while QaRL, QUADS, and TRACE couple representations or realized rounding decisions [3, 6, 8, 13]. QUADS gives a particularly informative component comparison: asymmetric weight QAT recovers much of the learning loss, and rollout-side residual compensation improves the remaining gap. Its asymmetric-versus-symmetric QAT comparison further separates nominal symmetry from effective alignment. FP8-RL adds the system perspective, showing how calibration and synchronization support a changing low-precision policy [4]. Together, these studies locate the benefit in a specified interface. Transfer depends on whether another stack preserves that interface and the surrounding computation.

Signed update selection explains effects hidden by averages. The full-pipeline FP8 study connects excessive lower-side clipping of negative-advantage tokens to pathological training dynamics, whereas TRIAGE uses local and direction-dependent information to select interventions [11, 12]. Both link stability to the signed signal retained after control. An aggregate clipping fraction or mean log-probability gap can hide that structure. TRIAGE's MoE routing-conditioned analysis also shows limits to token-wise directional predictiveness, distinguishing a population pattern from a reliable predictor for each token.

Table 4: Representative within-study observations supporting the synthesis. Settings and interpretation are retained beside each result; Appendix A summarizes the full focal corpus.

Study / setting	Published comparison or observation	What the evidence supports
QuRL [5] INT8/FP8 rollout	Longer-run behavior differs from early TIS behavior; ACR and invariant-scaling ablations address distinct failure modes.	Correction magnitude, clipping semantics, and update visibility are separate concerns; a short stable window is insufficient.
QUADS [8] NVFP4 MoE rollout	Weight-only learner QAT recovers much of the loss; adding rollout activation compensation further improves held-out scores.	The two components have complementary evidence within this design; neither universal necessity nor cross-engine transfer follows.
Score Centering [10] BF16 learner; quantization or lag	Shared-objective experiments compare additive centering and importance correction; their composition helps in the reported staleness tests.	Estimator drift and importance-weight control are distinguishable. The implemented behavior expectation is approximated.
Full Pipeline FP8 [11] FP8 learner and rollout	Calibrated Clipping improves the reported learning outcomes across scaling choices. Main GRPO results use best-checkpoint selection over 500 steps.	Restoring signed update contributions can matter; the selected endpoint is not a fixed-budget final-checkpoint comparison.
TRIAGE [12] Native NVFP4	Local tail patterns motivate direction-selective gating and repair. Token-wise predictiveness has a narrower scope than the population characterization.	Local and signed diagnostics can be informative, with limits under MoE coupling; token-independent contraction is not established.
KV4-RL [17] INT4 KV; partial collection	Unprotected INT4 KV misses later learning gains under the retained TIS recipe; bounded INT8 protection recovers near-BF16 endpoint quality.	Conditional correction and cache fidelity address different effects. Timing and quality claims apply to the studied collection workload.

The studies converge on the learning signal preserved by the combined design. Weight alignment and activation compensation address complementary errors in QUADS; UAQ and ACR connect update visibility with clipping in QuRL; cache protection recovers later gains under an unchanged correction recipe in KV4-RL. These examples explain why better execution can reduce the burden on correction, while a better update rule can make residual execution error tolerable.

6.3. Efficiency Gains and Implementation Costs

The same complementarity has a system cost. For serialized phases, iteration time can be decomposed as

$$T_{\text{iter}} = T_{\text{rollout}} + T_{\text{learner}} + T_{\text{sync}} + T_{\text{quant/calib}} + T_{\text{other}}, \tag{11}$$

where synchronization and quantization/calibration costs are separated from generation and learning. Accelerating one phase exposes the others. With overlapping execution, the relevant quantity is the critical path, so the phase times in Equation (11) cannot simply be added.

The serialized model also gives a simple bound on what a rollout improvement can achieve. If rollout occupies fraction f of the baseline time, a rollout speedup s and added work hT_0 give $T/T_0 = (1-f) + f/s + h$, holding the other phases and workload fixed. Figure 10 illustrates the arithmetic. Overlap, changed response lengths, or changed concurrency require a different cost model.

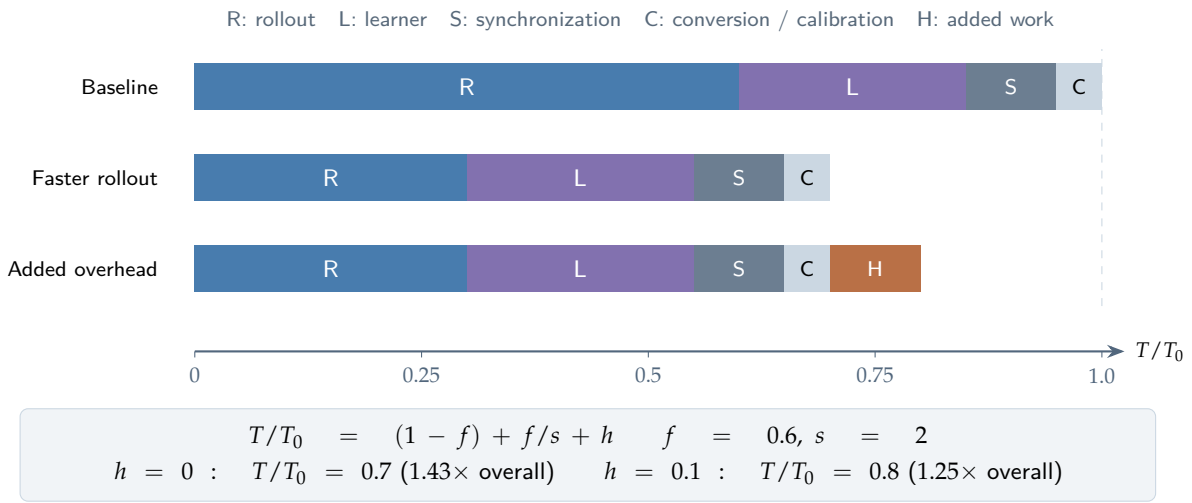


Figure 10: Analytical timing illustration for serialized execution. Doubling rollout speed reduces iteration time from T_0 to $0.7T_0$ when rollout accounts for 60% of the baseline. Added work of $0.1T_0$ reduces the overall speedup from 1.43× to 1.25×. These are constructed values.

Each intervention changes a different part of this cost. FP8-RL illustrates the work needed for scale estimation and numerical conversion [4]. Reference calibration adds learner computation; guided rounding adds storage, transport, and state matching; selective precision adds work in the protected region. Table 5 relates these dependencies to the information each method needs.

Token throughput also depends on the realized workload. A policy that produces shorter responses can appear faster without improving execution per token; one that produces longer responses can incur more work despite a faster kernel. Memory savings can increase concurrency, which changes the workload seen by attention and compute kernels. Serving research such as PagedAttention provides background for why cache management and scheduling affect observed throughput [23], but serving speed alone does not establish RL learning efficiency.

Table 5: Information and execution dependencies of the reviewed approaches. These interfaces explain system cost without assigning an overhead ranking.

Approach	Information or execution dependency	Consequence for implementation
TIS / AIS / CIS	Behavior probabilities and learner/proximal probabilities	Stored probabilities must include the sampling transformations.
Score Centering	A behavior-weighted score expectation, exact or approximated	Exact evaluation requires distributional information; reconstruction introduces approximation.
Aligned QAT	A learner path corresponding to rollout quantization	Unmatched kernels and interfaces leave residual execution error.
Guided rounding	Realized rollout-side quantization information	Guidance requires storage, transport, and matching to learner states.
Calibrated Clipping [11]	Higher-precision reference ratios at calibration points	Reference passes add training computation, amortized across calibration intervals.
Selective KV precision	Low-bit cache execution with protected positions	Protected positions add work whose share depends on the context workload.

KV4-RL illustrates the difference between generation and training gains. Over 400 iterations on Qwen3-8B-Base, it reports 1.80× cumulative rollout speedup and 1.27× cumulative full-iteration speedup over BF16 rollout [17]. Other phases absorb part of the generation gain. Its common-BF16 evaluation separately compares learned-policy quality under a shared evaluation precision. Both conclusions apply to the study's collection workload.

Time or compute to a specified quality level connects these measurements to the learning objective. Endpoint scores, iteration time, and rollout throughput supply different parts of the picture. The trajectory is needed to recover time-to-quality when an intervention changes response length, learning speed, or checkpoint selection.

6.4. Limitations of Current Evidence

The strongest empirical support concerns mechanisms within the settings in which they were isolated. Model families, tasks, horizons, hardware, and numerical kernels vary across studies. Mathematical reasoning is a recurring evaluation setting, while interactive trajectories and broader task distributions are less directly covered. Transfer across precision formats likewise depends on scaling rules, higher-precision exceptions, and the surrounding kernels.

This synthesis inherits the scope of its curated sources and the implementation details they make available. Source-linked observations expose the basis of the comparisons, while unreported conditions remain unresolved. The collection supports neither an exhaustive account of the field nor independent replication of its empirical results.

7. Open Problems

Existing methods show that low-precision RL can improve through both numerical and learning interventions. The remaining challenge is to predict their joint behavior: when a discrepancy becomes harmful, which component addresses it, and how the benefit transfers across systems and tasks.

7.1. Predicting Harmful Mismatch

Mean disagreement is an incomplete predictor because learning depends on where error occurs and which update it changes. Confidence-conditioned tails, signed clipping distortions, conditional drift, and directional concentration offer more specific candidates. The unresolved step is to connect these quantities to future learning damage before a task score reveals it.

Different failure targets require different predictors. Collapse within a horizon, reduced progress per update, and failure to reach a quality threshold need not share the same precursor. A mechanism-based diagnostic would remain predictive after a change of model or correction rule. It would also explain intervention effects: lowering the diagnostic without improving learning may reflect correlation, or the suppression of useful signal together with the measured risk.

7.2. Composing Numerical and Learning Interventions

Numerical fidelity changes the operating regime of a correction. Conversely, a correction that suppresses updates can change the apparent value of finer quantization. Their joint design therefore depends on marginal benefit: how much additional learning a component preserves after another component has already acted.

Multiple masks, truncations, or trust bands can suppress the same contributions. Apparent stability may then reflect weak learning. The central problem is to characterize the joint estimator and the signed signal that survives, including the additional reference or distributional computation it requires. This would connect compatibility to quality gained per unit of computation.

7.3. Precision, Staleness, and Long-Horizon Interaction

Partial and asynchronous collection can make the sampling version token-dependent. A probability gap may then combine parameter change with execution mismatch, as Equation (1) makes explicit. Separating their effects requires information about snapshots, replay, or retained cache states that a production system may not store. The smallest behavior record sufficient for reliable correction remains a joint statistical and system question.

Long responses amplify this difficulty. An early numerical perturbation changes the prefixes visited later, even if conditional correction at each observed prefix is accurate. This suggests allocating precision according to position, layer, context, and training phase, with learning benefit as the objective. Local tensor error alone cannot identify where the extra fidelity is worth its cost.

Multi-turn interaction extends the same question beyond token prefixes. Tool observations and environment transitions add state that shapes later behavior and may need to enter the record alongside cache provenance and sampling probabilities. The taxonomy still locates numerical and learning interventions, but evidence from single-response generation leaves their effectiveness in these settings open.

7.4. Generalization and Comparable Evidence

Transfer hinges on assumptions about behavior and execution. A correction derived for sampling probabilities can change meaning when an engine returns pre-decoding log-probabilities; rounding guidance can lose correspondence after a synchronization change. Whether a diagnostic or intervention remains effective when these interfaces change is an open question. Controlled transfer across stacks would help distinguish a general mechanism from benefits tied to one kernel, collection protocol, or correction recipe.

8. Conclusion

Low-precision rollout places numerical approximation inside the learning process: it changes the policy that generates experience and can thereby change both the current update and future trajectories. The reviewed methods address this feedback through shared computation, preserved fidelity, estimator correction, update control, and schedule adaptation.

Their common design problem is to preserve useful learning signal at lower computational cost. Interface-specific alignment, evolving correction, and signed update behavior explain complementary parts of that problem. Predictive diagnostics and coherent composition can turn these findings into a broader account of efficient learning across numerical paths, models, and interaction settings.

A. Surveyed Papers and Section Mapping

Table 6 summarizes all twenty focal contributions, their numerical settings, operative components, and discussion locations. The year denotes first public appearance; the bibliography may cite a later formal publication. Core, supporting, and boundary roles describe their use in this review. A supporting study can still contain direct quantized-rollout experiments. Primary discussion locations appear first, followed by secondary locations in parentheses.

Table 6: Surveyed papers: numerical regimes, method components, and section mapping. Conference and preprint versions count as one contribution.

Work / first year	Numerical regime	Component(s) and contribution	Discussion
Core method and system studies (13)			
FlashRL / TIS [1] 2025	Higher-precision learner; quantized rollout	**B1**: truncated importance correction using recorded behavior probabilities.	5.1 (1.1; 2.1)
Jet-RL [3] 2026	Unified FP8 learner and rollout paths	**A1; A2**: coordinated precision flow and quantization granularity.	4.1 (6.2)
FP8-RL [4] 2026	W8A8 rollout; FP8 attention / KV; FP8 learner explored in v2	**A2; system**: scaling, calibration, and synchronization; adopts TIS/MIS.	4.2 (5.1; 6.2; 6.3)
QuRL [5] 2026	INT8 / FP8 rollout	**B2; A2**: adaptive clipping (ACR) and update-aware quantization (UAQ).	5.2 (3.1; 4.2; 5.4; 6.2)
QaRL [6] 2026	Quantized MoE rollout; aligned learner QAT	**A1; B2**: rollout-aligned QAT and sequence-level trust-band updates (TBPO).	4.1 (5.2; 5.4; 6.2)
AIS [7] 2026	BF16 learner / FP8 rollout; autoregressive and diffusion models	**B1**: adaptive mixture of uncorrected and importance-weighted estimators.	5.1 (6.1; 6.2)
QUADS [8] 2026	NVFP4 MoE rollout; learner weight QAT	**A1; A2**: asymmetric weight QAT and rollout residual activation compensation.	4.1 (4.2; 5.4; 6.2)
ACRL [9] 2026	FP8 rollout; discrepancy-dependent updates	**B2**: advantage-guided probability adjustment regulates discrepancy.	5.2 (3.3; 6.2)
Score Centering [10] 2026	BF16 learner; quantized rollout or policy lag	**B1**: additive score centering; behavior expectation is approximated.	5.1 (2.2; 3.2; 5.4; 6.2)
Full Pipeline FP8 RL [11] 2026	FP8 learner and rollout; calibration against BF16	**B2; system**: reference-calibrated clipping preserves signed contributions.	5.2 (3.2; 6.2; 6.3)
TRIAGE [12] 2026	Native NVFP4 W4A4; MoE analysis	**B2**: direction-selective gating, rebalancing, and bounded local repair.	5.2 (3.3; 6.2)
TRACE [13] 2026	FP4 MoE rollout; guided learner QAT	**A1**: transfers rollout quantization information to guide learner rounding.	4.1 (3.1; 6.2)
KV4-RL [17] 2026	INT4 KV with bounded INT8 protection	**A2; system**: protects sensitive cache positions under retained TIS; generation and optimization alternate.	4.2 (5.1; 6.2; 6.3)
Supporting formulation and general TIM studies (6)			
R3 [15] 2025	MoE routing; general execution mismatch	**A1 context**: replay of discrete routing choices; not uniquely quantization-induced.	3.1 (4.1)

Continued on next page

Table 6 continued: Surveyed papers

Work / first year	Numerical regime	Component(s) and contribution	Discussion
FP16-Mismatch [16] 2025	BF16 / FP16 numerical comparison	**A2 context**: precision-selection evidence; not a low-bit acceleration method.	3.1
MiniRL [14] 2025	Hybrid-engine RL; includes FP8-rollout experiments	**Formulation; B1/B2**: distinguishes behavior, proximal, and learner roles.	2.1 (1.2; 5.1)
Beyond Precision [18] 2026	General TIM; optimization scheduling	**B3**: mismatch-informed learning-rate scheduling; FP4 transfer remains open.	5.3 (3.3)
VeXact / Diagnosing TIM [19] 2026	Matched-execution diagnostic reference	**Diagnosis**: zero-mismatch counterfactual for attribution.	3.4
CIS [20] 2026	BF16 cross-engine mismatch	**B1 context**: confidence-dependent truncation; not direct FP4 validation.	5.1 (3.2)
Exploration boundary case (1)			
QeRL [2] 2025	NVFP4 with LoRA and adaptive noise	**Exploration boundary**: quantization noise can aid exploration in its low-rank setting.	Box 1

B. Source Selection and Version Handling

Construction of this snapshot. The collection comprises twenty focal studies selected for their contribution to quantized rollout, low-precision RL, or mechanisms needed to interpret training–rollout mismatch. Primary papers, official publication pages, and arXiv records provide the source material. This is a curated narrative review; systematic database retrieval and exhaustive screening were outside its scope.

Inclusion and exclusion. A focal study contributes a direct quantized-rollout or low-precision RL intervention, a mechanism needed to interpret that setting, or a carefully identified boundary case. General quantization and RL background is cited selectively. Papers devoted only to low-precision pretraining, unrelated generation modalities, or alternative rollout acceleration do not become focal entries solely because they use the same precision format.

Versions and names. The catalog distinguishes first-publication dates, publication venues, and reviewed versions. QeRL, QuRL, and QaRL use formal publication records in the bibliography. Conference and preprint versions, aliases, and changes in presentation count as one contribution. Unpinned arXiv links are distinguished from locally reviewed versions, since a later revision may change the claims supported by a source.

Attribution of evidence. Method claims and empirical observations are attributed to primary sources. Equations (1), (2), (4), (5), and (6) are explanatory identities under their stated assumptions. Figures and summary tables are original syntheses; constructed examples are identified separately from published measurements. The companion repository provides the catalog, bibliography, and source-linked evidence, with links to third-party papers.

C. Evidence, Assumptions, and Transfer

A result about mismatch is conditional on the policy that generated the data. Parameter versions, numerical paths, decoding transformations, and retained execution state determine the behavior law. Learner, proximal, and KL-reference policies define separate roles in correction and optimization. Their relationship fixes the meaning of a reported ratio or discrepancy.

The component taxonomy describes the intervention; numerical configuration describes the setting in which it acts. Conditional correction, truncated weights, partial execution alignment, and reconstructed expectations leave different residual approximations. An empirical result establishes the behavior of the tested combination under its evaluation conditions. Transfer requires the assumptions responsible for that result to remain valid.

References

[1] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, Jianfeng Gao. On the Rollout-Training Mismatch in Modern RL Systems. *OPT 2025: Optimization for Machine Learning*, 2025. https://www.opt-ml.org/papers/2025/paper116.pdf.

[2] Wei Huang, Yi Ge, Shuai Yang, Yicheng Xiao, Huizi Mao, Yujun Lin, Hanrong Ye, Sifei Liu, Ka Chun Cheung, Hongxu Yin, Yao Lu, Xiaojuan Qi, Song Han, Yukang Chen. QeRL: Quantization-enhanced Low-Rank Reinforcement Learning for LLMs. *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/a15a2ece7f0663d1ba7db91103ac61c9-Abstract-Conference.html.

[3] Haocheng Xi, Charlie Ruan, Peiyuan Liao, Yujun Lin, Han Cai, Yilong Zhao, Shuo Yang, Kurt Keutzer, Song Han, Ligeng Zhu. Jet-RL: Enabling On-Policy FP8 Reinforcement Learning with Unified Training and Rollout Precision Flow. *arXiv preprint*, 2026. https://arxiv.org/abs/2601.14243.

[4] Zhaopeng Qiu, Shuang Yu, Jingqi Zhang, Shuai Zhang, Xue Huang, Jingyi Yang, Junjie Lai. FP8-RL: A Practical and Stable Low-Precision Stack for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2601.18150.

[5] Yuhang Li, Reena Elangovan, Xin Dong, Priyadarshini Panda, Brucek Khailany. QuRL: Efficient Reinforcement Learning with Quantized Rollout. *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/6b4880ced785077159b340710ceb7681-Abstract-Conference.html.

[6] Hao Gu, Hao Wang, Jiacheng Liu, Lujun Li, Qiyuan Zhu, Bei Liu, Binxing Xu, Lei Wang, Xintong Yang, Sida Lin, Sirui Han, Yike Guo. QaRL: Rollout-Aligned Quantization-Aware RL for Fast and Stable Training under Training–Inference Mismatch. *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. https://aclanthology.org/2026.findings-acl.841/.

[7] Jiajun Zhou, Wei Shao, Lingchao Zheng, Yuwei Fan, Ngai Wong. AIS: Adaptive Importance Sampling for Quantized RL. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.13907.

[8] Zhengyang Zhuge, Hao Yu, Xin Wang, Zheng Li, Yizhong Cao, Dayiheng Liu, Jianwei Zhang. QUADS: Stabilizing NVFP4 Reinforcement Learning for MoE via QUantization-error Alignment across Dual Sides. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.15810.

[9] Wenwu Fan, Qihong Lin, Zhijie Xia, Zhuo Zheng, Sihao Wang, Qiang Chen, Liangsheng Zhu. ACRL: Adaptive Control of Training-Inference Discrepancy for Stable Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.24062.

[10] Martin Marek, Max Ryabinin. Score Centering Stabilizes Off-policy Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.20807.

[11] Fanchao Chen, Ziheng Jiang, Ziyun Wei, Zheng Zhong, Du Li, Chi Zhang, Haibin Lin, Shivaram Venkataraman. Towards Full Pipeline FP8 Reinforcement Learning for LLMs. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.22870.

[12] Zhen Li, Shuai Zhang, Yanggan Gu, Yiming Zhang, Yang Yu, Mingfa Feng, Congkai Xie, Shuang Yu, Junjie Lai, Hongxia Yang. TRIAGE: Direction-Aware Mismatch Stabilization of Native NVFP4 Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.07043.

[13] Xin Wang, Hao Yu, Zhengyang Zhuge, Bochao Mao, Zheng Li, Junda Feng, Yuyan Luo, Yi Zhang, Yizhong Cao, Mi Zhang, Dayiheng Liu, Jianwei Zhang. TRACE: Rollout-Guided Quantization-Aware Training for FP4 Reinforcement Learning of MoE Language Models. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.07767.

[14] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, Junyang Lin. Stabilizing Reinforcement Learning with LLMs: Formulation and Practices. *arXiv preprint*, 2025. https://arxiv.org/abs/2512.01374.

[15] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, Fuli Luo. Stabilizing MoE Reinforcement Learning by Aligning Training and Inference Routers. *arXiv preprint*, 2025. https://arxiv.org/abs/2510.11370.

[16] Penghui Qi, Zichen Liu, Xiangxin Zhou, Tianyu Pang, Chao Du, Wee Sun Lee, Min Lin. Defeating the Training-Inference Mismatch via FP16. *arXiv preprint*, 2025. https://arxiv.org/abs/2510.26788.

[17] Sihwan Park, Sung-Yub Kim, Doohyuk Jang, Eunho Yang. KV4-RL: Unlocking Faster Asynchronous Rollouts for RLVR with INT4 KV Cache Quantization. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.14448.

[18] Yaxiang Zhang, Yingru Li, Jiacai Liu, Jiawei Xu, Ziniu Li, Qian Liu, Haoyuan Li. Beyond Precision: Training-Inference Mismatch is an Optimization Problem and Simple LR Scheduling Fixes It. *arXiv preprint*, 2026. https://arxiv.org/abs/2602.01826.

[19] Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, Xiao Yu. Diagnosing Training Inference Mismatch in LLM Reinforcement Learning via a Zero-Mismatch Reference. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.14220.

[20] Tianrun Yu, Kaixiang Zhao, Shangzhe Li, Yuxiao Yang, Porter Jenkins, Weitong Zhang, Taylor W. Killian. Rethinking Training-Inference Mismatch in LLM Reinforcement Learning: Where It Arises and How to Correct It. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.32444.

[21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint*, 2017. https://arxiv.org/abs/1707.06347.

[22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint*, 2024. https://arxiv.org/abs/2402.03300.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint*, 2023. https://arxiv.org/abs/2309.06180.

[24] Zhiwei Hao, Jianyuan Guo, Li Shen, Yong Luo, Han Hu, Guoxia Wang, Dianhai Yu, Yonggang Wen, Dacheng Tao. Low-Precision Training of Large Language Models: Methods, Challenges, and Opportunities. *arXiv preprint*, 2025. https://arxiv.org/abs/2505.01043.

[25] Rohan Surana, Gagan Mundada, Xunyi Jiang, Chuhan Wang, Zhenwei Tang, Difan Jiao, Zihan Huang, Yuxin Xiong, Junda Wu, Sheldon Yu, Xintong Li, Raghav Jain, Nikki Kuang, Sizhe Zhou, Bowen Jin, Zhendong Chu, Tong Yu, Ryan Rossi, Kuan-Hao Huang, Jingbo Shang, Jiawei Han, Julian McAuley. Generate, Filter, Control, Replay: A Comprehensive Survey of Rollout Strategies for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.02913.

[26] Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, Yu Fu, Xingtai Lv, Yuchen Zhang, Sihang Zeng, Shang Qu, Haozhan Li, Shijie Wang, Yuru Wang, Xinwei Long, Fangfu Liu, Xiang Xu, Jiaze Ma, Xuekai Zhu, Ermo Hua, Yihao Liu, Zonglin Li, Huayu Chen, Xiaoye Qu, Yafu Li, Weize Chen, Zhenzhao Yuan, Junqi Gao, Dong Li, Zhiyuan Ma, Ganqu Cui, Zhiyuan Liu, Biqing Qi, Ning Ding, Bowen Zhou. A Survey of Reinforcement Learning for Large Reasoning Models. *arXiv preprint*, 2025. https://arxiv.org/abs/2509.08827.